

TOTAL SECTIONS

53 Modules + Practice

CURRICULUM PART

Part 12 — Client-Side Storage & Persistence

CORE MECHANICS

localStorage, sessionStorage, Cookies & StorageEvent

REFERENCE STANDARD

W3C Web Storage / WHATWG / ECMAScript 2026

Browser Storage — Browser-এর মধ্যে Data Save, Read, Update ও Delete করা

একটি আধুনিক ওয়েব অ্যাপ্লিকেশনে ইউজার প্রফারেন্স (যেমন ডার্ক মোড, ভাষা), শপিং কার্ট ডাটা কিংবা সাময়িক ফর্ম ডাটা পেজ রিলোড বা ব্রাউজার বন্ধ করলেও যেন হারিয়ে না যায়, সেজন্য ব্রাউজারের নিজস্ব **Web Storage API** ব্যবহার করা হয়।

Client Application → Web Storage (localStorage / sessionStorage) / Cookies / IndexedDB → JSON Serialization → Persistent State

ব্রাউজারে নির্ভরযোগ্য স্টেট পারসিস্টেন্স এবং অফলাইন-রেডি ইউজার এক্সপেরিয়েন্সের মূল ভিত্তি হলো এই স্টোরেজ আর্কিটেকচার। 🔥

PART 01 — WEB STORAGE FOUNDATIONS & LOCALSTORAGE (18.1 – 18.10)**18.1 Browser Storage কী?**

Browser Storage হলো browser-এর মধ্যে website-এর কিছু data সংরক্ষণ করার ব্যবস্থা।

💡 REAL-LIFE EXAMPLE

ধরো তুমি একটি shopping website-এ গেলে এবং:

```
Dark Mode → ON  
Language → বাংলা  
Cart → 3 products
```

Browser কিছু information মনে রাখতে পারে।

তাই পরেরবার website খুললে:

```
Dark Mode → ON  
Language → বাংলা
```

থাকতে পারে।

18.2 Web Storage API

JavaScript-এর Web Storage API-এর প্রধান দুটি অংশ:

```
localStorage  
sessionStorage
```

দুটোই সাধারণত **key-value pair** আকারে data রাখে।

```
key           value  
-----  
username      Shariar  
theme         dark  
language      bn
```

18.3 localStorage কী?

`localStorage` browser-এ data save করে এবং সাধারণত browser বন্ধ করলেও data থেকে যায়।

18.3 JAVASCRIPT

RUN

```
localStorage.setItem("username", "Shariar");
```

এখন browser-এ:

TEXT

```
username = Shariar
```

save হয়েছে।

18.4 localStorage.setItem()

Syntax:

18.4 JAVASCRIPT

RUN

```
localStorage.setItem(key, value);
```

Example:

18.4 JAVASCRIPT

RUN

```
localStorage.setItem("name", "Shariar");
```

আরেকটি:

18.4 JAVASCRIPT

RUN

```
localStorage.setItem("theme", "dark");
```

18.5 localStorage.getItem()

Data পড়তে:

18.5 JAVASCRIPT

RUN

```
const name = localStorage.getItem("name");  
  
console.log(name);
```

TEXT

```
Shariar
```

18.6 localStorage.removeItem()

একটি নির্দিষ্ট item delete:

18.6 JAVASCRIPT

RUN

```
localStorage.removeItem("name");
```

এখন:

18.6 JAVASCRIPT

RUN

```
console.log(localStorage.getItem("name"));
```

TEXT

null

18.7 localStorage.clear()

সব localStorage data delete করতে:

18.7 JAVASCRIPT

RUN

```
localStorage.clear();
```

⚠ এটি ওই origin-এর localStorage-এর সব entries মুছে দিতে পারে।

তাই production code-এ সাবধানে ব্যবহার করতে হবে।

18.8 localStorage.length

কতগুলো item আছে:

18.8 JAVASCRIPT

RUN

```
console.log(localStorage.length);
```

যদি থাকে:

```
name  
theme  
language
```

তাহলে:

TEXT

3

18.9 localStorage.key()

Index দিয়ে key পাওয়া যায়।

18.9 JAVASCRIPT

RUN

```
console.log(localStorage.key(0));
```

Output উদাহরণ:

TEXT

name

⚠ Key-এর order-এর উপর application logic নির্ভর করানো উচিত নয়।

18.10 localStorage-এর সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য

```
localStorage
├── Key → Value
├── Persistent
├── Same origin
└── String-based
```

সবচেয়ে গুরুত্বপূর্ণ:

localStorage মূলত string data store করে।

PART 02 — DATA SERIALIZATION: NUMBERS, BOOLEANS, OBJECTS & JSON FORMULA (18.11 – 18.21)

18.11 Number Save করলে কী হয়?

18.11 JAVASCRIPT

RUN

```
localStorage.setItem("age", 25);
```

পরে:

18.11 JAVASCRIPT

RUN

```
const age = localStorage.getItem("age");

console.log(age);
console.log(typeof age);
```

TEXT

```
25
string
```

অর্থাতঃ:

```
25 Number
  ↓
localStorage
  ↓
"25" String
```

18.12 String থেকে Number

18.12 JAVASCRIPT

RUN

```
const age = Number(localStorage.getItem("age"));

console.log(age);
console.log(typeof age);
```

TEXT

```
25
number
```

18.13 Boolean Save করলে?

18.13 JAVASCRIPT

RUN

```
localStorage.setItem("isLoggedIn", true);
```

পরে:

18.13 JAVASCRIPT

RUN

```
const value = localStorage.getItem("isLoggedIn");  
  
console.log(value);  
console.log(typeof value);
```

TEXT

true
string

⚠ এটি Boolean `true` নয়, string `"true"`।

18.14 Boolean Convert করা

18.14 JAVASCRIPT

RUN

```
const isLoggedIn = localStorage.getItem("isLoggedIn") === "true";  
  
console.log(isLoggedIn);
```

TEXT

true

18.15 Object কি সরাসরি Save করা যায়?

ধরো:

18.15 JAVASCRIPT

RUN

```
const user = {  
  name: "Shariar",  
  age: 24,  
};
```

এভাবে করলে:

18.15 JAVASCRIPT

RUN

```
localStorage.setItem("user", user);
```

সমস্যা হবে।

পরে:

18.15 JAVASCRIPT

RUN

```
console.log(localStorage.getItem("user"));
```

Output হতে পারে:

TEXT

[object Object]

কারণ Web Storage string-based।

18.16 Object Save করার জন্য `JSON.stringify()

18.16 JAVASCRIPT

RUN

```
const user = {  
  name: "Shariar",  
  age: 24,  
  role: "Developer",  
};  
  
localStorage.setItem("user", JSON.stringify(user));
```

এখন object string JSON format-এ save হয়েছে।

18.17 Object Read করার জন্য `JSON.parse()

18.17 JAVASCRIPT

RUN

```
const storedUser = localStorage.getItem("user");  
  
const user = JSON.parse(storedUser);  
  
console.log(user);
```

18.17 JAVASCRIPT

RUN

```
{  
  name: "Shariar",  
  age: 24,  
  role: "Developer"  
}
```

এখন:

18.17 JAVASCRIPT

RUN

```
console.log(user.name);
```

TEXT

Shariar

18.18 JSON + localStorage Formula

এটা খুব ভালোভাবে মনে রাখো:

⚙️ SAVE:

18.18 JAVASCRIPT

RUN

```
localStorage.setItem("user", JSON.stringify(user));
```

⚙️ READ:

18.18 JAVASCRIPT

RUN

```
const user = JSON.parse(localStorage.getItem("user"));
```

★ MENTAL MODEL

```
Object
  ↓
JSON.stringify()
  ↓
String
  ↓
localStorage
```

Read করার সময়:

```
localStorage
  ↓
String
  ↓
JSON.parse()
  ↓
Object
```

18.19 Array Save করা

18.19 JAVASCRIPT

RUN

```
const skills = ["HTML", "CSS", "JavaScript", "React"];

localStorage.setItem("skills", JSON.stringify(skills));
```

Read:

18.19 JAVASCRIPT

RUN

```
const skills = JSON.parse(localStorage.getItem("skills"));

console.log(skills);
```

TEXT

```
["HTML", "CSS", "JavaScript", "React"]
```

18.20 Array-এর মধ্যে নতুন Data যোগ করা

ধরো:

```
18.20 JAVASCRIPT RUN
const skills = JSON.parse(localStorage.getItem("skills"));

skills.push("Node.js");

localStorage.setItem("skills", JSON.stringify(skills));
```

এখন storage-এ:

```
TEXT
HTML
CSS
JavaScript
React
Node.js
```

18.21 Safe JSON Parsing

যদি storage-এ কিছু না থাকে:

```
18.21 JAVASCRIPT RUN
const user = JSON.parse(localStorage.getItem("user"));
```

`getItem()` যদি `null` দেয়, তাহলে `JSON.parse(null)` JavaScript-এ `null` দেয়, কিন্তু malformed JSON থাকলে error হতে পারে।

Safe pattern:

```
18.21 JAVASCRIPT RUN
const storedUser = localStorage.getItem("user");

let user = null;

if (storedUser) {
  try {
    user = JSON.parse(storedUser);
  } catch (error) {
    console.error("Invalid stored user data", error);
  }
}

console.log(user);
```

PART 03 — SESSIONSTORAGE & REAL-WORLD THEME/DARK MODE STATE (18.22 – 18.27)

18.22 sessionStorage কী?

`sessionStorage` -ও key-value storage।

```
18.22 JAVASCRIPT RUN
sessionStorage.setItem("username", "Shariar");
```

Read:

```
18.22 JAVASCRIPT RUN
console.log(sessionStorage.getItem("username"));
```

18.23 localStorage vs sessionStorage

Feature	localStorage	sessionStorage
Data format	String	String
Persistent	Yes	Session-based
Tab-specific	No	Yes
Browser restart	Usually remains	Typically cleared when the page session ends
API	Same	Same
<code>setItem()</code>	Yes	Yes
<code>getItem()</code>	Yes	Yes
<code>removeItem()</code>	Yes	Yes
<code>clear()</code>	Yes	Yes

সহজভাবে

```
localStorage  
→ "পরের বারও মনে রাখো"  
  
sessionStorage  
→ "এই tab/session-এর জন্য রাখো"
```

18.24 sessionStorage Example

ধরো একটি checkout process:

```
Step 1 → Personal Info  
Step 2 → Address  
Step 3 → Payment
```

Current tab-এর session-এর মধ্যে temporary state রাখতে:

```
18.24 JAVASCRIPT  
sessionStorage.setItem("checkoutStep", "2");
```

RUN

পরে:

```
18.24 JAVASCRIPT  
console.log(sessionStorage.getItem("checkoutStep"));
```

RUN

TEXT

2

18.25 Theme Switcher — Real-life Example

এখন localStorage-এর একটি খুব common use case দেখি।

HTML:

18.25 HTML

MARKUP

```
<button id="themeBtn">Toggle Theme</button>
```

JavaScript:

18.25 JAVASCRIPT

RUN

```
const button = document.querySelector("#themeBtn");

button.addEventListener("click", function () {
  document.body.classList.toggle("dark");

  const isDark = document.body.classList.contains("dark");

  localStorage.setItem("theme", isDark ? "dark" : "light");
});
```

Page reload হলেও theme preference পাওয়া যাবে।

18.26 Page Load-এর সময় Theme Restore

শুধু save করলে হবে না। Page load হলে read করতে হবে।

18.26 JAVASCRIPT

RUN

```
const savedTheme = localStorage.getItem("theme");

if (savedTheme === "dark") {
  document.body.classList.add("dark");
}
```

Complete Flow

```
User selects Dark Mode
    ↓
localStorage.setItem()
    ↓
Page reload
    ↓
localStorage.getItem()
    ↓
Dark Mode restore
```

18.27 Dark Mode Example

HTML:

18.27 HTML

MARKUP

```
<button id="themeBtn">Dark Mode</button>
```

CSS:

TEXT

```
.dark {  
  background: #111;  
  color: white;  
}
```

JavaScript:

18.27 JAVASCRIPT

RUN

```
const button = document.querySelector("#themeBtn");  
  
const savedTheme = localStorage.getItem("theme");  
  
if (savedTheme === "dark") {  
  document.body.classList.add("dark");  
}  
  
button.addEventListener("click", function () {  
  document.body.classList.toggle("dark");  
  
  const theme = document.body.classList.contains("dark") ? "dark" : "light";  
  
  localStorage.setItem("theme", theme);  
});
```

ଏটি real-world website-এ খুব common pattern।

18.28 Shopping Cart with localStorage

ধরো:

18.28 JAVASCRIPT

RUN

```
const cart = [
  {
    id: 1,
    name: "Laptop",
    price: 70000,
  },
  {
    id: 2,
    name: "Mouse",
    price: 1500,
  },
];
```

Save:

18.28 JAVASCRIPT

RUN

```
localStorage.setItem("cart", JSON.stringify(cart));
```

Read:

18.28 JAVASCRIPT

RUN

```
const cartData = JSON.parse(localStorage.getItem("cart")) || [];
console.log(cartData);
```

এখানে:

18.28 JAVASCRIPT

RUN

```
|| []
```

এর কারণে storage-এ cart না থাকলে empty array পাওয়া যাবে।

18.29 Add Product to Cart

18.29 JAVASCRIPT

RUN

```
const product = {
  id: 3,
  name: "Keyboard",
  price: 2500,
};

const cart = JSON.parse(localStorage.getItem("cart")) || [];

cart.push(product);

localStorage.setItem("cart", JSON.stringify(cart));
```

18.30 Remove Product from Cart

18.30 JAVASCRIPT

RUN

```
let cart = JSON.parse(localStorage.getItem("cart")) || [];

cart = cart.filter((product) => product.id !== 2);

localStorage.setItem("cart", JSON.stringify(cart));
```

18.31 localStorage দিয়ে User Preferences

একটি website-এ:

18.31 JAVASCRIPT

RUN

```
const settings = {  
  theme: "dark",  
  language: "bn",  
  fontSize: "large",  
};  
  
localStorage.setItem("settings", JSON.stringify(settings));
```

Read:

18.31 JAVASCRIPT

RUN

```
const settings = JSON.parse(localStorage.getItem("settings"));  
  
console.log(settings.language);
```

TEXT

bn

18.32 Storage-এ Sensitive Data রাখা যাবে?

এখানে খুব গুরুত্বপূর্ণ security concept আছে।

localStorage -এ sensitive information রাখা risky হতে পারে।

বিশেষ করে:

TEXT

Password
Long-lived secret
Highly sensitive personal information
Authentication secrets

কারণ JavaScript একই origin-এর storage access করতে পারে এবং XSS vulnerability থাকলে attacker storage-এর data পড়তে পারে।

তাই:

✗ Password localStorage-এ রেখো না।

18.32 JAVASCRIPT

RUN

```
localStorage.setItem("password", "12345678");
```

এটা করা উচিত নয়।

18.33 Authentication Token নিয়ে সতর্কতা

অনেক web application authentication-এর জন্য token ব্যবহার করে।

যেমন:

TEXT

Access Token
Refresh Token

এগুলো কোথায় রাখা হবে—এটি security architecture-এর গুরুত্বপূর্ণ বিষয়।

বিশেষ করে authentication token localStorage-এ রাখার আগে XSS risk বিবেচনা করতে হয়।

Sensitive authentication cookie-এর ক্ষেত্রে server-managed:

TEXT

HttpOnly
Secure
SameSite

cookie attributes ব্যবহার করা যায়।

18.34 Cookies কী?

Cookie হলো browser-এ ছোট data store করার একটি mechanism।

Example:

```
theme=dark
language=bn
```

JavaScript দিয়ে:

18.34 JAVASCRIPT

RUN

```
document.cookie = "theme=dark";
```

Read:

18.34 JAVASCRIPT

RUN

```
console.log(document.cookie);
```

TEXT

```
theme=dark
```

18.35 Cookie vs localStorage

Feature	localStorage	Cookie
Storage	Client-side	Client-side
JavaScript access	Yes	Usually yes, unless HttpOnly
Automatically sent with HTTP requests	No	Yes, subject to cookie rules
Size	তুলনামূলক বেশি	ছোট
Expiration	সাধারণত explicit expiration নয়	Expiration সেট করা যায়
Server access	Directly নয়	Request-এর সাথে পাঠানো যায়
Common use	Preferences, local app state	Session/auth-related state, server-managed preferences

18.36 Cookie তৈরি করা

18.36 JAVASCRIPT

RUN

```
document.cookie = "theme=dark; max-age=3600; path="/;
```

এখানে:

TEXT

```
theme=dark
max-age=3600 seconds
path=/
```

18.37 Cookie Delete

একটি cookie expire করতে:

18.37 JAVASCRIPT

RUN

```
document.cookie = "theme=; max-age=0; path="/;
```

18.38 Cookie-এর Important Attributes

Max-Age

TEXT

```
max-age=3600
```

কত seconds পরে expire করবে।

Expires

নির্দিষ্ট expiry time দেওয়া যায়।

Path

TEXT

```
path=/  

```

কোন path-এ cookie applicable।

Secure

HTTPS connection-এর মাধ্যমে cookie পাঠাতে নির্দেশ দেয়।

HttpOnly

JavaScript থেকে cookie access বন্ধ করে।

⚠️ `HttpOnly` cookie JavaScript দিয়ে `document.cookie` থেকে পড়া যায় না।

SameSite

Cross-site request-এর ক্ষেত্রে cookie behavior নিয়ন্ত্রণে সাহায্য করে।

18.39 IndexedDB কী?

`localStorage` ছোট/simple data-এর জন্য ভালো।

কিন্তু যদি বড় বা structured client-side data রাখতে হয়, তখন:

TEXT

```
IndexedDB
```

ব্যবহার করা যায়।

এটি browser-এর একটি **asynchronous, transactional, structured database API**।

18.40 IndexedDB কখন ব্যবহার করব?

ধরো তুমি বানাচ্ছে:

TEXT

```
Offline Notes App  
Offline Video Metadata App  
Large Client-side Dataset  
Offline-first Web App
```

এখানে হাজার হাজার structured record থাকতে পারে।

তখন `localStorage` ideal নয়।

18.41 localStorage vs IndexedDB

বিষয়	localStorage	IndexedDB
Data type	String	Structured data
API	Simple	More advanced
Async	No	Yes
Large data	Not suitable	Better
Transactions	No	Yes
Queries/indexes	Very limited	Supported
Complexity	Easy	More complex

18.42 IndexedDB-এর Basic Idea

Conceptually:

```
Database
  ↓
Object Store
  ↓
Records
```

উদাহরণ:

```
Database: StudentDB

Object Store: students

1 → Shariar
2 → Rahim
3 → Karim
```

IndexedDB asynchronous হওয়ায় এর code localStorage-এর তুলনায় বেশি complex।

PART 06 — STORAGE EVENTS, UTILITY WRAPPER & ENTERPRISE BEST PRACTICES (18.43 – 18.49)

18.43 Storage Events

একটি interesting browser event হলো:

```
18.43 JAVASCRIPT
window.addEventListener("storage", function (event) {
  console.log(event.key);
  console.log(event.oldValue);
  console.log(event.newValue);
});
```

RUN

`storage` event সাধারণত অন্য same-origin document/context-এ storage পরিবর্তন হলে useful হয়।

18.44 Storage Event Example

ধরো একই website-এর দুইটি tab খোলা:

```
Tab A      Tab B
|          |
|  localStorage.set  |
|----->|
|          |
|          | storage event
```

Tab B পরিবর্তন detect করতে পারে।

18.45 Storage Data Inspect করা

Chrome DevTools-এ:

```
DevTools
↓
Application
↓
Storage
↓
Local Storage
```

এখানে দেখতে পারবে:

Key	Value
theme	dark
username	Shariar
cart	[...]

এটি debugging-এর জন্য খুব usefull

18.46 localStorage Utility Function

বারবার code না লিখে function তৈরি করতে পারো।

18.46 JAVASCRIPT

RUN

```
function saveData(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
```

Use:

18.46 JAVASCRIPT

RUN

```
saveData("user", {
  name: "Shariar",
  age: 24,
});
```

18.47 Generic Read Function

18.47 JAVASCRIPT

RUN

```
function getData(key) {
  const data = localStorage.getItem(key);

  if (!data) {
    return null;
  }

  try {
    return JSON.parse(data);
  } catch (error) {
    console.error("Invalid JSON:", error);

    return null;
  }
}
```

Use:

18.47 JAVASCRIPT

RUN

```
const user = getData("user");

console.log(user);
```

এটি real project-এ utility হিসেবে ব্যবহার করা যায়।

18.48 Storage Wrapper

আরও সুন্দরভাবে:

18.48 JAVASCRIPT

RUN

```
const storage = {
  set(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
  },

  get(key) {
    const value = localStorage.getItem(key);

    if (value === null) {
      return null;
    }

    try {
      return JSON.parse(value);
    } catch {
      return null;
    }
  },

  remove(key) {
    localStorage.removeItem(key);
  },

  clear() {
    localStorage.clear();
  },
};
```

Use:

18.48 JAVASCRIPT

RUN

```
storage.set("user", {
  name: "Shariar",
  role: "Developer",
});
```

Read:

18.48 JAVASCRIPT

RUN

```
console.log(storage.get("user"));
```

18.49 Storage-এর Limitation

localStorage খুব useful হলেও সবকিছুর জন্য নয়।

⚠ LIMITATIONS:

1. String-based
2. Synchronous API
3. Limited storage capacity
4. Same-origin scoped
5. XSS risk
6. Not suitable for large databases
7. Not suitable for secrets

MUST KNOW 🔥 18.50 Must Know — Storage Engine Security & Mechanics

এই chapter থেকে অবশ্যই জানতে হবে:

JAVASCRIPT

RUN

```
localStorage.setItem();
```

JAVASCRIPT

RUN

```
localStorage.getItem();
```

JAVASCRIPT

RUN

```
localStorage.removeItem();
```

JAVASCRIPT

RUN

```
localStorage.clear();
```

JAVASCRIPT

RUN

```
localStorage.length;
```

JAVASCRIPT

RUN

```
localStorage.key();
```

JSON Storage

JAVASCRIPT

RUN

```
JSON.stringify();
```

JAVASCRIPT

RUN

```
JSON.parse();
```

Session

JAVASCRIPT

RUN

```
sessionStorage.setItem();
```

JAVASCRIPT

RUN

```
sessionStorage.getItem();
```

Cookies

JAVASCRIPT

RUN

```
document.cookie;
```

Advanced Storage

TEXT

IndexedDB

কাজ	Code
Save	<code>localStorage.setItem("key", "value")</code>
Read	<code>localStorage.getItem("key")</code>
Delete one	<code>localStorage.removeItem("key")</code>
Delete all	<code>localStorage.clear()</code>
Count	<code>localStorage.length</code>
Get key	<code>localStorage.key(index)</code>
Object → String	<code>JSON.stringify(obj)</code>
String → Object	<code>JSON.parse(str)</code>
Session save	<code>sessionStorage.setItem()</code>
Session read	<code>sessionStorage.getItem()</code>
Cookie	<code>document.cookie</code>
Large structured data	<code>IndexedDB</code>



Practice 1 — Save User Name

Input থেকে name নিয়ে:

```
localStorage
```

এ save করো।

Page reload করার পর name আবার দেখাও।

Practice 2 — Dark Mode

একটি button বানাও:

```
🌙 Dark Mode  
☀️ Light Mode
```

User preference localStorage-এ save করো।

Practice 3 — Shopping Cart

এই structure ব্যবহার করো:

```
JAVASCRIPTRUN  
  
{  
  id: 1,  
  name: "Laptop",  
  price: 70000  
}
```

Features:

```
Add Product  
Remove Product  
Clear Cart  
Cart Count  
Total Price
```

সব data localStorage-এ রাখো।

Practice 4 — Todo App

Features:

```
TEXT  
  
Add Todo  
Delete Todo  
Mark Complete  
Clear All
```

Page reload করলেও todo যেন থাকে।

Practice 5 — Student Profile

Save:

```
JAVASCRIPTRUN  
  
{  
  name: "Shariar",  
  studentId: "23105101019",  
  department: "CSE",  
  university: "Daffodil International University"  
}
```

localStorage-এ save করে reload-এর পর UI-তে দেখাও।

Practice 6 — Session Storage

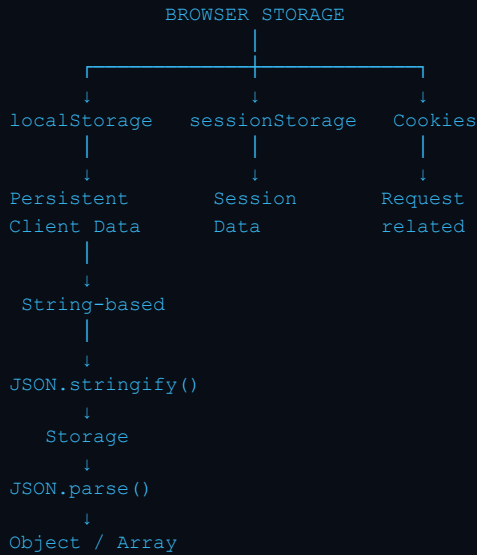
একটি multi-step form বানাও:

```
Step 1 → Personal Information
Step 2 → Academic Information
Step 3 → Confirmation
```

Current step `sessionStorage`-এ রাখো।

SUMMARY

🗺️ 18.53 Final Mental Map — Client Storage Decision Architecture



আর বড় structured offline data হলে:

```
Large / Structured Data
↓
IndexedDB
```

★ ★ CHAPTER 18-এর সবচেয়ে গুরুত্বপূর্ণ CONCEPT

তুমি যদি শুধু একটি জিনিস মনে রাখতে চাও:

```
JAVASCRIPT
const user = {
  name: "Shariar",
  role: "Developer",
};

// Save
localStorage.setItem("user", JSON.stringify(user));

// Read
const savedUser = JSON.parse(localStorage.getItem("user"));

console.log(savedUser);
```

এই pattern বুঝে ফেললে **localStorage + JSON + object/array persistence**-এর মূল concept তোমার পরিষ্কার হয়ে যাবে।

Chapter 18 complete.

NEXT CHAPTER

Chapter 19 Preview — JSON (JavaScript Object Notation)

পরবর্তী অধ্যায়ে আমরা আধুনিক ওয়েব কমিউনিকেশন ও ডেটা ইন্টারচেঞ্জের বিশ্বজনীন ফরম্যাট **JSON (JavaScript Object Notation)** নিয়ে গভীরভাবে শিখব— `JSON.stringify()`, `JSON.parse()`, `replacer` ও `reviver` ফাংশন, `deep clone`, `circular reference` হ্যান্ডলিং এবং API ডেটা ফরম্যাটিং।